

Stellar Payments SDK (Web + Mobile) — Monorepo Skeleton

Below is a complete, ready-to-copy scaffold for your GitHub repository. It includes folder structure, key files, CI, and a minimal working SDK for **Phase 1** (Web/TypeScript + React component). Replace `yourorg` with your GitHub org/user.

Folder Tree

```
stellar-payments-sdk/  
├── .github/  
│   └── workflows/  
│       └── ci.yml  
├── .gitignore  
├── LICENSE  
├── README.md  
├── package.json  
├── tsconfig.json  
├── tsup.config.ts  
├── src/  
│   ├── index.ts  
│   ├── types.ts  
│   ├── core/  
│   │   ├── createPaymentSession.ts  
│   │   └── signAndSubmit.ts  
│   └── components/  
│       └── PayWithUSDC.tsx  
├── examples/  
│   └── nextjs-demo/  
│       ├── package.json  
│       ├── pages/  
│       │   └── index.tsx  
│       └── next.config.js  
└── .npmrc
```

package.json (root)

```
{  
  "name": "@yourorg/payments-sdk",
```

```

"version": "0.1.0",
"description": "Universal Payments SDK for Stellar (USDC) – Web + Mobile",
"license": "MIT",
"repository": {
  "type": "git",
  "url": "git+https://github.com/yourorg/stellar-payments-sdk.git"
},
"bugs": {
  "url": "https://github.com/yourorg/stellar-payments-sdk/issues"
},
"homepage": "https://github.com/yourorg/stellar-payments-sdk#readme",
"type": "module",
"exports": {
  ".": {
    "types": "./dist/index.d.ts",
    "import": "./dist/index.js",
    "require": "./dist/index.cjs"
  }
},
"main": "./dist/index.cjs",
"module": "./dist/index.js",
"types": "./dist/index.d.ts",
"files": [
  "dist"
],
"scripts": {
  "build": "tsup src/index.ts --dts --format esm,cjs --sourcemap",
  "dev": "tsup src/index.ts --watch --sourcemap",
  "lint": "echo 'add eslint later'",
  "release": "npm publish --access public"
},
"peerDependencies": {
  "react": ">=18.0.0",
  "react-dom": ">=18.0.0"
},
"dependencies": {
  "stellar-sdk": "^12.2.0"
},
"devDependencies": {
  "@types/react": "^18.3.0",
  "@types/react-dom": "^18.3.0",
  "typescript": "^5.6.0",
  "tsup": "^8.0.0"
}
}

```

Note: The `exports` field sets correct ESM/CJS and **prevents the tsup warning** about `types` ordering.

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "declaration": true,
    "declarationMap": true,
    "outDir": "dist",
    "strict": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "lib": ["ES2020", "DOM"],
    "types": ["react", "react-dom"]
  },
  "include": ["src"]
}
```

tsup.config.ts

```
import { defineConfig } from 'tsup'

export default defineConfig({
  entry: ['src/index.ts'],
  dts: true,
  clean: true,
  format: ['esm', 'cjs'],
  sourcemap: true,
  target: 'es2020',
  minify: false,
  splitting: false,
})
```

.gitignore

```
node_modules
.dist
coverage
.DS_Store
*.log
dist
```

LICENSE (MIT)

MIT License

Copyright (c) 2025 Your Org

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

[...standard MIT text continues...]

src/types.ts

```
export type Network = 'testnet' | 'public'

export type PayWithUSDCProps = {
  amount: string | number
  destination: string
  network?: Network
  memo?: string
  onSuccess?: (txHash: string) => void
  onError?: (err: unknown) => void
}
```

src/core/createPaymentSession.ts

```
import { Keypair, Networks, TransactionBuilder, Operation, Asset, Account,
Memo } from 'stellar-sdk'

export type CreateSessionInput = {
  amount: string | number
  destination: string
  sourcePublicKey: string
  sequence: string
  network: 'testnet' | 'public'
  memo?: string
}

export async function createPaymentSession(input: CreateSessionInput) {
  const { amount, destination, sourcePublicKey, sequence, network, memo } =
input

  const account = new Account(sourcePublicKey, sequence)
  const USDC = new Asset('USDC', 'GA5ZSEK...ISSUER') // TODO: inject issuer via
config

  const tx = new TransactionBuilder(account, {
    fee: '100',
    networkPassphrase: network === 'public' ? Networks.PUBLIC :
Networks.TESTNET,
  })
  tx.addOperation(
    Operation.payment({
      destination,
      asset: USDC,
      amount: String(amount),
    })
  )
  tx.addMemo(memo ? Memo.text(memo) : Memo.none())
  tx.setTimeout(120)
  tx.build()

  return tx
}
```

src/core/signAndSubmit.ts

```
import { Server, Keypair, Networks, Transaction } from 'stellar-sdk'

export async function signAndSubmit(params: {
  serverUrl?: string
  tx: Transaction
  secretKey: string
  network: 'testnet' | 'public'
}) {
  const { serverUrl, tx, secretKey, network } = params

  const server = new Server(
    serverUrl || (network === 'public' ? 'https://horizon.stellar.org' :
'https://horizon-testnet.stellar.org')
  )
  const keypair = Keypair.fromSecret(secretKey)

  tx.sign(keypair)
  const res = await server.submitTransaction(tx)
  return res.hash
}
```

src/components/PayWithUSDC.tsx

```
import React, { useState } from 'react'
import { PayWithUSDCProps } from '../types'
import { Server } from 'stellar-sdk'
import { createPaymentSession } from '../core/createPaymentSession'
import { signAndSubmit } from '../core/signAndSubmit'

export function PayWithUSDC(props: PayWithUSDCProps) {
  const { amount, destination, memo, onSuccess, onError, network = 'testnet' }
= props
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)

  async function handlePay() {
    try {
      setLoading(true)
      setError(null)

      // In a real app, source account comes from connected wallet (e.g.,
```

```

Freighter)
  const secretKey = (window as any).__STELLAR_SECRET__ // ⚠ For demo only
  const server = new Server(network === 'public' ? 'https://
horizon.stellar.org' : 'https://horizon-testnet.stellar.org')
  const keypair = require('stellar-sdk').Keypair.fromSecret(secretKey)

  const account = await server.loadAccount(keypair.publicKey())
  const tx = await createPaymentSession({
    amount,
    destination,
    memo,
    sourcePublicKey: keypair.publicKey(),
    sequence: account.sequence,
    network,
  })

  const hash = await signAndSubmit({ tx, secretKey, network })
  onSuccess?.(hash)
} catch (e) {
  setError((e as Error).message)
  onError?.(e)
} finally {
  setLoading(false)
}
}

return (
  <button disabled={loading} onClick={handlePay}>
    {loading ? 'Processing...' : `Pay ${amount} USDC`}
  </button>
)
}

```

src/index.ts

```

export * from './types'
export * from './core/createPaymentSession'
export * from './core/signAndSubmit'
export * from './components/PayWithUSDC'

```

examples/nextjs-demo/package.json

```
{
  "name": "nextjs-demo",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "14.2.4",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "@yourorg/payments-sdk": "link:../../"
  }
}
```

examples/nextjs-demo/pages/index.tsx

```
import React, { useEffect } from 'react'
import { PayWithUSDC } from '@yourorg/payments-sdk'

export default function Home() {
  useEffect(() => {
    // DEMO ONLY: inject a test secret for quickstart (replace with wallet
    integration)
    ;(window as any).__STELLAR_SECRET__ = 'SA...TEST_SECRET...'
  }, [])

  return (
    <div style={{ padding: 32 }}>
      <h1>Next.js + Stellar Payments SDK</h1>
      <PayWithUSDC amount={"10"} destination="GDESTINATION..."
network="testnet" />
    </div>
  )
}
```


examples/nextjs-demo/next.config.js

```
/** @type {import('next').NextConfig} */
const nextConfig = {}
module.exports = nextConfig
```

README.md (root)

```
# Stellar Payments SDK (Web + Mobile)

**One-line checkout SDK to make paying with USDC on Stellar as easy as Stripe.**

## Features (Phase 1)
- `
```

```
import { PayWithUSDC } from '@yourorg/payments-sdk'

<PayWithUSDC
  amount="50"
  destination="GDESTINATION..."
  network="testnet"
  onSuccess={(h) => console.log('ok', h)}
  onError={(e) => console.error(e)}
/>
```

Dev

```
npm i
npm run build
```

Examples

```
examples/nextjs-demo
```

Roadmap

- Phase 2: Backend webhooks + SEP-24 integration
- Phase 3: React Native package

License

MIT

```
---

## 🟠 CI — .github/workflows/ci.yml
```.yaml
name: ci
on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 build:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 cache: 'npm'
 - run: npm ci
 - run: npm run build
```

---

## .npmrc (optional for provenance / provenance warnings)

```
provenance=true
```

---

## How to Use

1. **Create repo:** `stellar-payments-sdk` on GitHub.
2. Paste this scaffold, commit, push.
3. `npm ci && npm run build`.

4. Link the example: `cd examples/nextjs-demo && npm i && npm run dev`.
5. Swap the **USDC issuer** in `createPaymentSession.ts` via config (env or constructor) before real tests.



## Publish to npm

```
npm version patch
npm publish --access public
```

For org-scoped packages, ensure the org is allowed to publish public packages.



## Next Steps (Phase 1 hardening)

- Replace demo `window.__STELLAR_SECRET__` with **Freighter** or Wallet SDK connect.
- Externalize **USDC issuer** + network via config.
- Add **unit tests** for transaction building.
- Improve error surfaces and retry/backoff. ``